MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Manoj Mishra

Subject & Subject Code: Software Engineering 3CS4-05

Branch: CSE

Batch: A & B

Semester: 3

Session: 2022-23(odd Sem.)

Unit No.: 3

Date of Submission: 1-11-2023

Lect.No.	Topics Covered	Page No.	Total pages
1	Requirement Analysis task	1-4	4
2	Software prototyping and specification	5-7	3
3	Data Dictionary	8-9	2
4	Finite State Machine Model(FSM)	10-17	8
5	Data flow diagram	18-21	4
6	Control flow diagram	22-29	8
7	Control and process specification	30-34	5
8	Behavioral modeling	35-47	13

References Book:

1. Software engineering 3rd edition K.K Aggarwal
2. Fundamental of software Engineering,pearson by Rajib Mall,

FACULTY HOD DEAN ACADEMICS

2/11/23

Scanned by Scanner Go

LECTURE NO:-1

Requirment Analysis

In systems engineering and software engineering, requirements analysis focuses on the tasks that determine the requirements or conditions to satisfy the new or altered product or project, taking account of the possibly conflicting requirements of the varied stakeholders, analyzing, documenting, validating and managing software or system requirements.

Requirements analysis is critical to the success or failure of a systems or software project.The requirements should be documented, actionable, measurable, testable, traceable, associated with identified business needs or opportunities, and defined to A level of detail sufficient for system design.

Conceptually, requirements analysis includes three sorts of activities:

- Eliciting requirements: (e.g. the project charter or definition), business process documentation, and stakeholder interviews. This is sometimes also called requirements gathering or requirements discovery.
- Recording requirements: Requirements could also be documented in various forms, usually including a summary list and should include naturallanguage documents, use cases, user stories, process specifications and a spread of models including data models.
- Analyzing requirements: determining whether the stated requirements are clear, complete, unduplicated, concise, valid, consistent and unambiguous, and resolving any apparent conflicts. Analyzing can also include sizing requirements.

Requirements analysis are often an extended and tiring process during which many delicate psychological skills are involved. New systems change the environment and relationships between people, so it's important to spot all the

stakeholders, take under consideration all their needs and ensure they understand the implications of the new systems.

Analysts can employ several techniques to elicit the wants from the customer. These may include the event of scenarios (represented as user stories in agile methods), the identification of use cases, the utilization of workplace

MTRC

observation or ethnography, holding interviews, or focus groups (more aptly named during this context as requirements workshops, or requirements review sessions) and creating requirements lists. Prototyping could also be wont to

develop an example system which will be demonstrated to stakeholders. Where necessary, the analyst will employ a combination of these methods to establish the exact requirements of the stakeholders, so that a system that meets the business needs is produced.[citation needed] Requirements quality can be improved through these and other methods

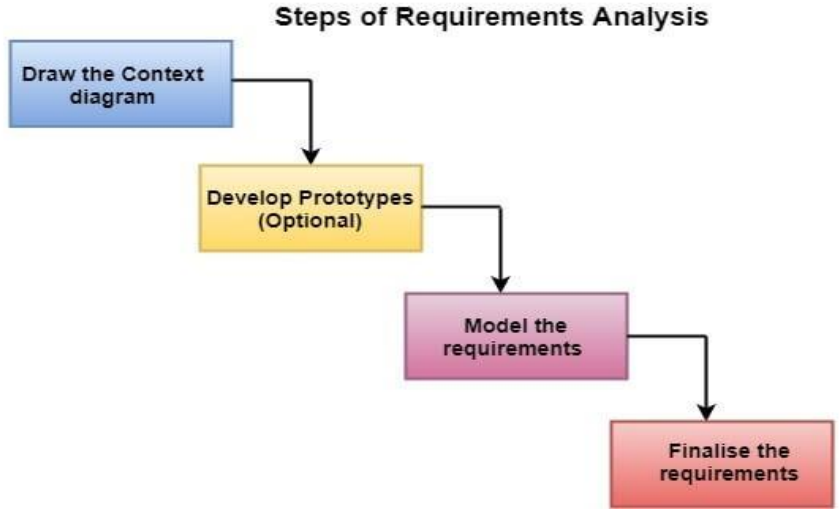
•**Visualization.** Using tools that promote better understanding of the desired end-product such as visualization and simulation.

•**Consistent use of templates.** Producing a consistent set of models and templates to document the requirements.

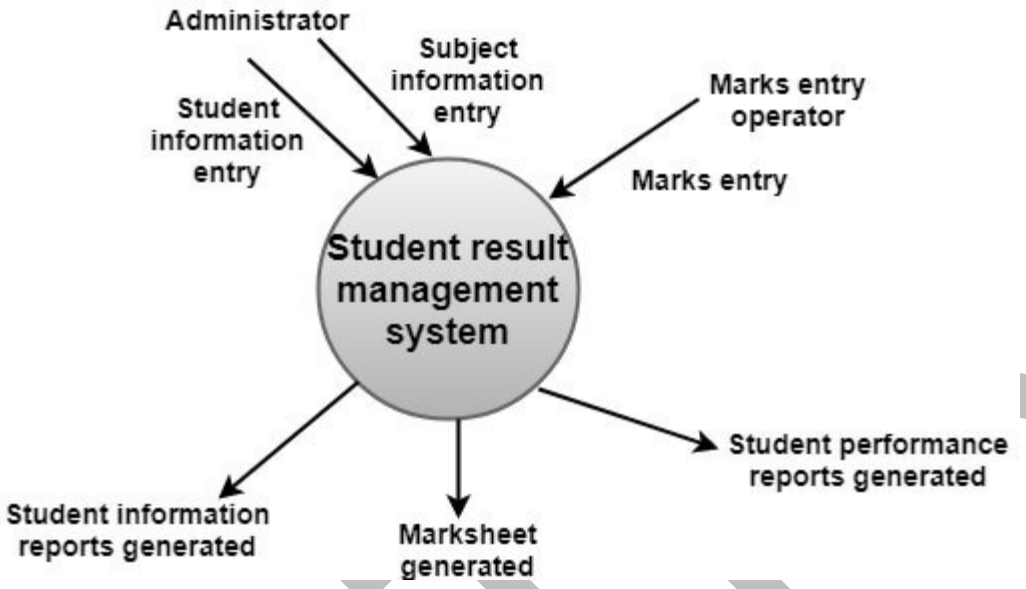
•**Documenting dependencies.** Documenting dependencies and interrelationships among requirements, as well as any assumptions and congregations.

Requirement analysis is significant and essential activity after elicitation. We analyze, refine, and scrutinize the gathered requirements to make consistent and unambiguous requirements. This activity reviews all requirements and may provide a graphical view of the entire system. After the completion of the analysis, it is expected that the understandability of the project may improve significantly. Here, we may also use the interaction with the customer to clarify points of confusion and to understand which requirements are more important than others.

The various steps of requirement analysis are shown in fig:



(i) Draw the context diagram: The context diagram is a simple model that defines the boundaries and interfaces of the proposed systems with the external world. It identifies the entities outside the proposed system that interact with the system. The context diagram of student result management system is given below:



(ii) Development of a Prototype (optional): One effective way to find out what the customer wants is to construct a prototype, something that looks and preferably acts as part of the system they say they want.

We can use their feedback to modify the prototype until the customer is satisfied continuously. Hence, the prototype helps the client to visualize the proposed system and increase the understanding of the requirements. When developers and users are not sure about some of the elements, a prototype may help both the parties to take a final decision.

Some projects are developed for the general market. In such cases, the prototype should be shown to some representative sample of the population of potential purchasers. Even though a person who tries out a prototype may not buy the final system, but their feedback may allow us to make the product more attractive to others.

The prototype should be built quickly and at a relatively low cost. Hence it will always have limitations and would not be acceptable in the final system. This is an optional activity.

(iii) Model the requirements: This process usually consists of various graphical representations of the functions, data entities, externalentities, and the relationships between them. The graphical view may help to find incorrect, inconsistent, missing, and superfluous requirements. Such models include the Data Flow diagram, Entity-Relationship diagram, Data Dictionaries, State-transition diagrams, etc.

(iv) Finalise the requirements: After modeling the requirements, we will have a better understanding of the system behavior. The inconsistencies and ambiguities have been identified and corrected. The flow of data amongst various modules has been analyzed. Elicitation and analyze activities have provided better insight into the system. Now we finalize the analyzed requirements, and the next step is to document these requirements in a prescribed format.

MITRC

LECTURE NO:-2

Requirment Analysis Task

In systems engineering and software engineering, requirements analysis focuses on the tasks that determine the requirements or conditions to satisfy the new or altered product or project, taking account of the possibly conflicting requirements of the varied stakeholders, analyzing, documenting, validating and managing software or system requirements.

Requirements analysis is critical to the success or failure of a systems or software project.The requirements should be documented, actionable, measurable, testable, traceable, associated with identified business needs or opportunities, and defined to A level of detail sufficient for system design.

Conceptually, requirements analysis includes three sorts of activities:

- Eliciting requirements: (e.g. the project charter or definition), business process documentation, and stakeholder interviews. This is sometimes also called requirements gathering or requirements discovery.
- Recording requirements: Requirements could also be documented in various forms, usually including a summary list and should include naturallanguage documents, use cases, user stories, process specifications and a spread of models including data models.
- Analyzing requirements: determining whether the stated requirements are clear, complete, unduplicated, concise, valid, consistent and unambiguous, and resolving any apparent conflicts. Analyzing can also include sizing requirements.

Requirements analysis are often an extended and tiring process during which many delicate psychological skills are involved. New systems change the environment and relationships between people, so it's important to spot all the

stakeholders, take under consideration all their needs and ensure they understand the implications of the new systems.

Analysts can employ several techniques to elicit the wants from the customer. These may include the event of scenarios (represented as user stories in agile methods), the identification of use cases, the utilization of workplace

MTRC

observation or ethnography, holding interviews, or focus groups (more aptly named during this context as requirements workshops, or requirements review sessions) and creating requirements lists. Prototyping could also be wont to

develop an example system which will be demonstrated to stakeholders. Where necessary, the analyst will employ a combination of these methods to establish the exact requirements of the stakeholders, so that a system that meets the business needs is produced.[citation needed] Requirements quality can be improved through these and other methods

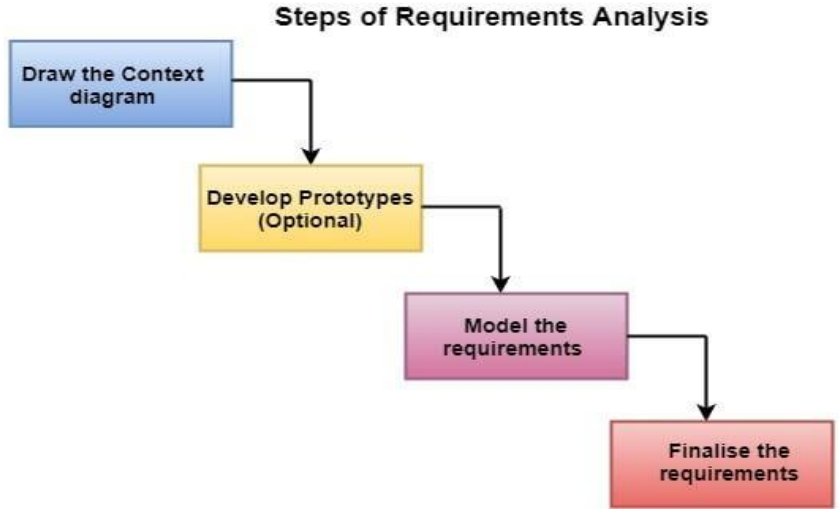
•**Visualization.** Using tools that promote better understanding of the desired end-product such as visualization and simulation.

•**Consistent use of templates.** Producing a consistent set of models and templates to document the requirements.

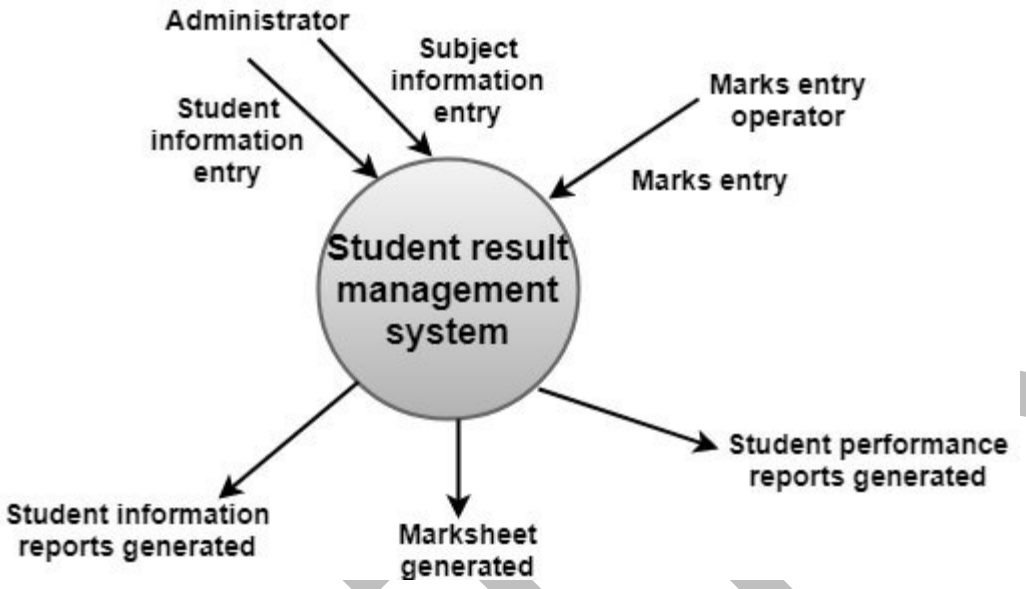
•**Documenting dependencies.** Documenting dependencies and interrelationships among requirements, as well as any assumptions and congregations.

Requirement analysis is significant and essential activity after elicitation. We analyze, refine, and scrutinize the gathered requirements to make consistent and unambiguous requirements. This activity reviews all requirements and may provide a graphical view of the entire system. After the completion of the analysis, it is expected that the understandability of the project may improve significantly. Here, we may also use the interaction with the customer to clarify points of confusion and to understand which requirements are more important than others.

The various steps of requirement analysis are shown in fig:



(i) Draw the context diagram: The context diagram is a simple model that defines the boundaries and interfaces of the proposed systems with the external world. It identifies the entities outside the proposed system that interact with the system. The context diagram of student result management system is given below:



(ii) Development of a Prototype (optional): One effective way to find out what the customer wants is to construct a prototype, something that looks and preferably acts as part of the system they say they want.

We can use their feedback to modify the prototype until the customer is satisfied continuously. Hence, the prototype helps the client to visualize the proposed system and increase the understanding of the requirements. When developers and users are not sure about some of the elements, a prototype may help both the parties to take a final decision.

Some projects are developed for the general market. In such cases, the prototype should be shown to some representative sample of the population of potential purchasers. Even though a person who tries out a prototype may not buy the final system, but their feedback may allow us to make the product more attractive to others.

The prototype should be built quickly and at a relatively low cost. Hence it will always have limitations and would not be acceptable in the final system. This is an optional activity.

(iii) Model the requirements: This process usually consists of various graphical representations of the functions, data entities, externalentities, and the relationships between them. The graphical view may help to find incorrect, inconsistent, missing, and superfluous requirements. Such models include the Data Flow diagram, Entity-Relationship diagram, Data Dictionaries, State-transition diagrams, etc.

(iv) Finalise the requirements: After modeling the requirements, we will have a better understanding of the system behavior. The inconsistencies and ambiguities have been identified and corrected. The flow of data amongst various modules has been analyzed. Elicitation and analyze activities have provided better insight into the system. Now we finalize the analyzed requirements, and the next step is to document these requirements in a prescribed format.

MITRC

LECTURE NO:-3

Requirment Analysis Principles

Analysis Principles

Over the past two decades, a large number of analysis modelling methods have been developed

By Analyzing Problems And Their Causes, investigator have developed a variety of
- notations
- correspondig set of heuristic overcome.

Each analysis method has a unique point of view, however all analysis methods are related by a set of operational principles:

1) The information domain of a problem must be represented and understood.

2) The functions that the software is to perform must be defined.

3) The behavior of the software must be represented.

4) The Models that depict information and behavior must be partitioned in a manner that uncovers detail in a layered (or Hierarchical) fashion.
5) the analysis process should move from essential information towards implementation detail.

By applying this principles, an analyst approaches a problem systematically.

A Set of guiding principles fro Requirement Engineering:

- **Understand** the problem before you begin to create the analysis model.
- **Develop Prototype** that enable a user to understand how human / machine interaction will occur.
- **Record** the origin of and reason for every requirement.
- **Use Multiple use of requirements-** building data, functional and behavioral models provide the software engineering with different 3 views.
- **Rank Requirements:** Tight deadline may preclude the implementation of every software requirement. If an incremental process model is applied, those requirements to be delivered in the first increment must be identified.
- **Work to Eliminate Ambiguity:** As most requirements are described in

MITRC

natural language, the opportunity for everybody abounds. The use of formal technical review is one way to uncover and eliminate ambiguity.

MITRC

LECTURE NO:-4

Software Prototyping

Software Design Fundamentals are given following:

1. The Software Design Process

1.1 Definitions and Aspects

Software design is the technical kernel of the software engineering process.

Software design is a process through which the requirements are translated into a representation of software. Initially the representation depicts a holistic view of software. Subsequent refinement leads to a representation that is close to source code.

The software design process has technical and management aspects:

a) from a project management point of view, software design is conducted in two steps:

- preliminary design: concerns the transformation of requirements into data and software architecture;

- detailed design: focuses on refinements to the architectural representation that lead to detailed data structure and algorithmic representations of software;

b) from a technical point of view, software design involves the following steps:

- data design: transforms the information domain model created during the analysis into interaction.

data structures that will be required to implement the software;

- architectural design: defines the relationship among major structural components of the program;

- procedural design: transforms the structural components into a procedural description of the software;

- interface design: establishes the layout and interaction mechanisms for human-machine

1.2 Fundamental Concepts

The software designer raises the following questions:

MITRC

- what criteria can be used to partition the software into individual components?

- how is function or data structure detail separated from a conceptual representation of the software?

- are there uniform criteria that define technical quality of software design?

The basic concepts of the software design process are:

- abstraction

- refinements

- modularity

- architecture

- hierarchy

- data structure

- software procedures

- information hiding

2. Object-Oriented vs. Procedural Design

2.1. Procedural Design

The procedural design is often understood as a software design process that uses mainly control commands such as: sequence, condition, repetition, which are applied to the predefineddata.

Sequences serve to achieve the processing steps in order that is essential in the specification of any algorithm.

Conditions provide facilities for achieving selected processing according to some logical statement.

Repetitions serve to achieve loopings during the computation process.

These three commands are implemented as ready programming language constructs.

The programming languages that provide such command constructs are called imperative programming languages.

The software design technique that relies on these constructs is called procedural design, or also structured design.

MITRC

2.2. Object-Oriented Design

2.2.1. The Macro Design Process

The macro design process is closely related to the **traditional waterfall life cycle**, and serves as controlling framework for the micro process.

The basic philosophy of the macro process is that of incremental development.

The macro design process tends to track the following activities:

Conceptualization: includes establishing the core requirements to the software.

The conceptualization may consist of the following activities:

- assemble an appropriate team to develop a prototype;

- develop a prototype;

- evaluate the prototype.

The prototypes constructed at this stage are meant to be thrown away. Such prototypes are not to be allowed to evolve directly into a system. At this stage any programming paradigm may be used.

Analysis: the purpose of the analysis is to provide a description of a problem.

The description must be complete, consistent, readable, and reviewable. The primary activities during the analysis are:

- identify all primary function points of the system, if possible group them

into clusters of functionally related behaviours; Function points are the outwardly observable and testable behaviours of the system. From the perspective of the user, a function point represents some primary activity of the system in response to some event.

- for each set of function points storyboard a scenario using use-cases;

- develop the use-case view of the system.

During the analysis we seek to model the world by classes from the vocabulary of the problem domain.

MITRC

Design: the purpose of the design is to create the architecture of the system The primary activities during the design are:

- allocate the clusters of function points to layers and partitions of the architecture;

- perform one spin of corresponding micro design subprocesses !

- validate the architecture by creating an executable release;

- asses the risk of each architectural component.

After the global system construction, release planning sets. A typical order of events for this activity is as follows:

- given the architectural components organize them in order of behaviors;

- allocate the relevant function points to series of architectural releases; Elaborate the Component Diagram and develop the Component view of the system;

- develop the Deployment Diagram and the Deployment view of the system.

- perform task planning, when the work breakdown structure is ready. A product of release planning is a formal development plan, which identifies the stream of architectural releases, team tasks, and risk assessments.

Evolution: this is essentially the process of implementing the system

The primary activities during the evolution process are:

- assign tasks to the team to carry out this release;

- initiate one spin of corresponding micro design subprocesses !

- supervise the micro process by establishing appropriate reviews;

- force closure of the micro process by making an executable release.

In practice the following kinds of changes are to be expected during the system evolution:

- adding new classes to the system;

- changing the implementation of a class;

- changing the representation of a class;

- changing the interface of a class.

Maintenance: this is the activity of managing postdelivery evolution of the system.

The primary activities during the maintenance are:

- prioritize the requests for major enhancement or bug reports that denote systematic problems, and asses the cost of development;

- establish a meaningful collection of these changes and treat them as function points for next elaboration;

- if resources allow it, add more localized enhancements to the next release;

- manage the next evolutionary release.

2.2.2 The Micro Design Process

The micro design process is closely related to the **spiral model** of software development, and serves as framework for iterative and incremental approach to development.

Identifying Classes and Objects: the purpose is to establish the boundaries of the problem.

The activities at this process are:

- apply the classical approach to object-oriented development to generate a set of candidate classes and objects. The abstractions should be directly related to the function points.

- develop Class and Object Diagrams ! For each abstraction do the following:

- enumerate its roles and responsibilities;

- devise a sufficient set of operations that satisfy these responsibilities;

- consider each operation in tern and ensure that it is primitive. If not, isolate and expose its more primitive operations.

- consider the needs for construction, destruction and copying;

MITRC

- consider the need for completeness: add other primitive operations that are not necessarily required by the immediate clients, and would probably be used by future clients.

Identifying the Semantics of Classes and Objects: the purpose is to establish the behavior and attributes of each abstraction. Here we refine the abstractions through distribution of the responsibilities.

A typical order of events might be the following:

- create a scenario or scenarios related to a single function point

- walk through the activities of the scenarios, assigning responsibilities to each abstraction sufficient to accomplish the desired behavior. If necessary, assign attributes that represent structural elements required to carry out certain responsibilities.

- reallocate responsibilities so that there is a reasonably balanced distribution of behavior. Where possible reuse or adapt existing responsibilities. Splitting large responsibilities to smaller ones is very common action.

- elaborate the State Diagrams and Sequence Diagrams.

Identifying the Relationships among Classes and Objects: the purpose is to solidify the boundaries of and to recognize the collaborations with each abstraction. This activity formalizes the conceptual as well as physical separations of concern among the abstractions.

The activities night be:

- collect a set of classes at a given level of abstraction and add the significant properties relevant to the problem being modeled;

- establish association relationships in case of presented semantic dependencies; Some associations may immediately be identified as specialization/generalization or aggregation relationships;

- for each association specify the role of each participant, as well as any relevant cardinality or other kind of constraint;

- validate the decisions by walk through scenarios;

MITRC

- develop Collaboration Diagrams.

Inheritance, containment, instantiation and use are the main kinds of relationships of interest, together with other properties as labels, cardinality, and so on:

- given a collection of classes clustered by some set of associations, look for patterns of behavior that represent opportunities for specialization/generalization. Place the classes in context of the inheritance lattice, or fabricate another one.

- if there are patterns of structure, consider creating new classes that capture this common structure, and introduce them either through inheritance or through aggregation;

- as development proceeds, introduce tactical details such as: roles, keys, cardinalities, friendship, and so on.

Implementing Classes and Objects: the purpose is to provide a refinement of the existing abstractions, sufficient to unveil new classes and objects at the next level of abstraction, which we feed at the next iteration. A typical order of events may be the following:

- for each class consider its protocol. Determine which operations need to be optimized. Develop precise signatures for all significant operations;

- adjust the inheritance lattice if required;

- alter local implementations of abstractions that are not time or space efficient;

- check whether the algorithms for the operations are suitable.

3. Design Documentation:

A **software design description** (a.k.a. **software design document** or **SDD**; just **design document**; also Software Design Specification) is a written description of a software product, that a software designer writes in order to give a software development team overall guidance to the architecture of the software project. An SDD usually accompanies an architecture diagram with pointers to detailed feature specifications of smaller pieces of the design. Practically, the description is

required to coordinate a large team under a single vision, needs to be a stable reference, and outline all parts of the software and how they will work.

The SDD usually contains the following information:

•The data design describes structures that reside within the software. Attributes and relationships between data objects dictate the choice of data structures.

•The architecture design uses information flowing characteristics, and maps them into the program structure. The transformation mapping method is applied to exhibit distinct boundaries between incoming and outgoing data. The data flow diagrams allocate control input, processing and output along three separate modules.

•The interface design describes internal and external program interfaces, as well as the design of the human interface. Internal and external interface designs are based on the information obtained from the analysis model.

•The procedural design describes structured programming concepts using graphical, tabular and textual notations.

These design mediums enable the designer to represent procedural detail, that facilitates translation to code. This blueprint for implementation forms the basis for all subsequent software engineering work.

MITRC

LECTURE NO:-5

Data Dictionary

A data dictionary is a file or a set of files that includes a database's metadata. The data dictionary hold records about other objects in the database, such as data ownership, data relationships to other objects, and other data. The data dictionary is an essential component of any relational database. Ironically, because of its importance, it is invisible to most database users. Typically, only database administrators interact with the data dictionary.

The data dictionary, in general, includes information about the following:

- Name of the data item
- Aliases
- Description/purpose
- Related data items
- Range of values & Data structure definition/Forms

Data dictionary is an essential and important tool in the software development process. As its name indicates, it is a comprehensive definition of all the data (and control) items (elements) in a given software. Or it can stated as—it is a structured central repository of data about data. In a data dictionary one should find a clear and complete definiton of each data item and its synonyms. The data dictionary often becomes a source document for specification and design of imput processing, files, and data structures, processing algorithms. and output processing. Typically, data dictionary development begins very early in the development process and expands as the software development process continues and more detail becomes available. To describe the data dictionary, sufficient information should be collected about data items. As such, there are a number of methods to collect the information about data items. In some methods, the data dictionary as a repository is not developed, and the related information about the data items is collected in the requirements, design and interface specifications only. In the other development methods, a separate document as a data dictionary is made to reside information. Format of Data Dictionary Althoug the format of data dictionaries varies from method to

MITRC

method, most use the following table 5.7, called as Project Data Element Sheet, and information about the data item.

Table 5.7 : Project Data Element Sheet

Name	Alias	Use	Content Description	Supplement/Additional Information

In this table 5.7 :

Name – a formal or primary name of the data or control item, data storage or external entity.

Alias – other names or acronyms of data item.

Use – a listing of processes which use the data or control item, and how or when it is used.

Content Description – standard format for representing the contents of data or control item.

Supplement/Additional Information – other information such as limitations, restrictions, defaults or initial value etc.

The description format is frequently given a highly structured form but it is to be noted that consistency in naming must be followed. Except the content description, all the information may be provided directly. To develop the content description of a data item, some notations (symbols) are used, as shown in table 5.8.

Table 5.8 : Content Description Symbols and Their Definitions

Symbol	Definition
=	is composed of
+	and
[|]	either/or
{ }^{n}	'n' repetitions of
( )	optional data
_ _ _	delimits comments

As an example, using the tables 5.7 and 5.8, a telephone number might appear in a data dictionary as, shown in table 5.9 :

Table 5.9 : Project Data Element Sheet of a Telephone Number

Name	Alias	Use	Content Description	Supplement/Additional Information
Telephone number	None	• Phone Setup • Dial phone	Telephone number = [local number/long distance number] local number = prefix + access number long distance number = 0 + area code + local number prefix = [2520/2524] access number = *any three–digit string* area code = [141, 151]	None

In this way, a data dictionary can be maintained.

Classes of Data Items

Basically, there are three classes of data items to be considered, as shown in fig.,

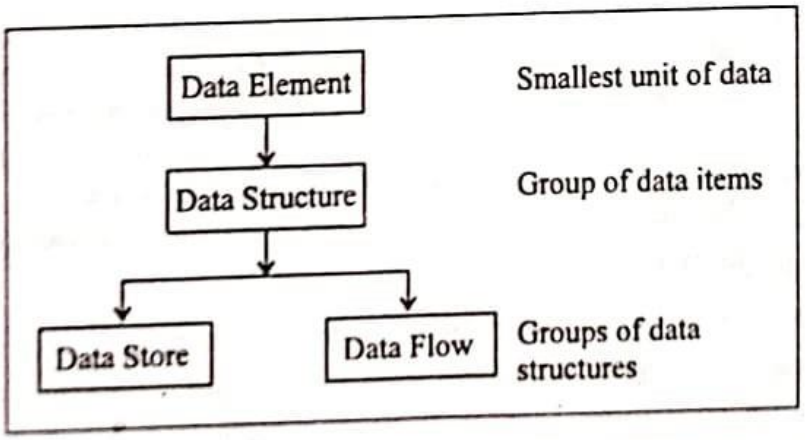


Fig. 5.15 : Logical Data Description Hierarchy

1. **Data Element :** The smallest unit of data that can not be further decomposed.
2. **Data Structure :** Once data items have defined, the data structure required by the particular system under consideration can be identified. The terms normally used to define data structures are :
   - **Group Items :** Combinations of data items for some logical purpose. For an example, the data item "Customer address" might well be combined with the data item "Customer name" to form a group item i.e. "Cusomter name and address".

• **Records :** Collections of data items or group items which are gathered to define an entity for a particular processing purpose. For an example, the data item "Customer number" and the group item "Cusomter name and address" might be brought together to form a customer name and address record: where by in a sales invoicing application, the customer number input to the system on a delivery note should be matched with the customer number in the customer name and address record to pick up the name and address.

• **Files :** Collections of records which are gathered together to allow processing of data about several entities. For an example, if in the invoicing system, all the deliveries for a particular day were processed to produce invoices then a file of customer name and address records would be required to match against all the different customers to whom a delivery had been made.

• **Database:** A collection of records which are interrelated for purposes of processing to meet user requirements. For an example, in a deliveries ditabase, the delivery record would be interrelated with the order record, product record, customer record and invoice record, in such a way as to minimize duplication of data.

3. Data Stores and Data Flows : Data stores and data flows are made up of data structures and elements. Consequently, data store and flow entires contain data structures and data elements. The data is these entries is described using any of the , above notations. Entires about data stores and flows also include other references. For example, the data flow description contains the origin and destination of the data flow. It also contains the physical description of the data flow, in this case stating that it takes theform of paper invoices. Data store descriptions also contain physical information, such as the medium used fo store the data and where it is located.

MITRC

LECTURE NO:-6

Finite State Machine Model

Requirement models are used to clarify and improve requirements consistency, unambiguity, correctness and completeness. The finite state machine (FSM) model is very popular with requirement engineers and developers, but all use the same basic model for requirements and implementation. This article presents a layered FSM model—a layer for each development phase, and presents the requirements layer in detail as the requirements engineer's tool for creating more correct and complete requirements. It also shows how this requirements model can guide design, implementation and testing, creating a requirements-derived development process.

Introduction

Requirement models are used to clarify and improve requirements consistency, unambiguity, correctness and completeness. Model types used in requirements engineering have visual representation and mathematical rigor; it is up to the requirements engineer to decide which of those two aspects to emphesize more. Some model types define data: entity-relationship diagrams that represent application entities and their relationships; class diagrams represent entities as classes, their data (members), operations (methods), and their inheritance and aggregation relationships. Process models include data flow diagrams that define processes (despite the name), their input and output data, how it flows between them and their hierarchical decomposition. Activity diagrams show processes and their temporal—sequential and concurrent relationships, and state transition diagrams or their more mathematical manifestation—finite state machines, that represent the dynamic behavior of entities in response to events. Here we will focus on that FSM model, examining its power to improve requirements' correctness and drive large parts of the following development steps.

The Requirements FSM

FSM's state definition is often quite general. All too often the state's name is the only definition used, and we trust it to convey all that the state entails. Sometimes a natural language description is added as part of the documentation. UML's notation includes state actions: entry, do and exit, but those can be viewed as design elements of the state, since they outline a structure or mechanism.

FSM Layers

In addition to requirements modeling, an FSM is often used for implementation, and since the blueprint of implementation is the design, it should be part of it too. We will indicate later how the FSM model can be used for testing. All those

models' information and code can be structured as a layered FSM and its state as Figure 1 depicts.

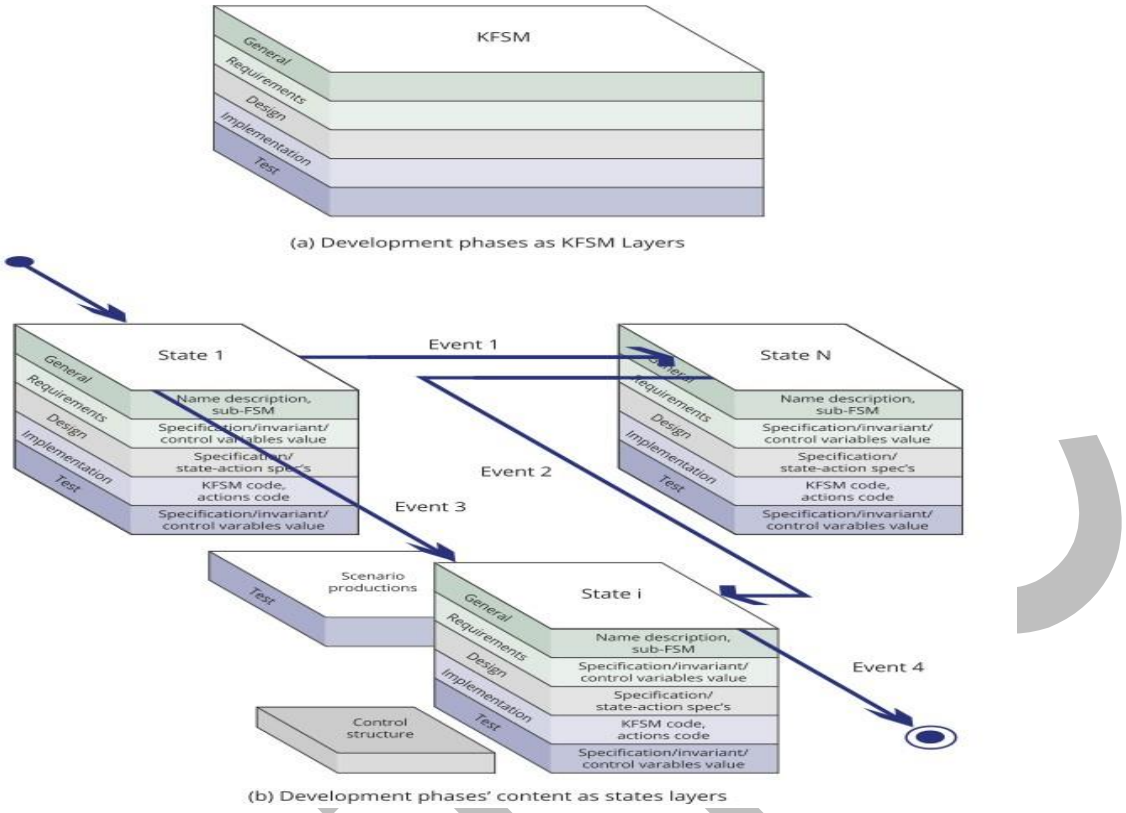


Figure 1: Development phase FSM layers

At the top – the General Layer, includes:

- **State transitions** – diagram or table, defining model's dynamics.
- **Control structure** – the set of all entities or properties that are affected or controlled by the FSM and its states (more details below).
- **States' general definition** – their name and description and attributes.

Next is the states' requirements layer, detailing the requirements for each state; it is the subject of this work and will be discussed in detail in the following sections. Combined, the general and requirements layers form the requirements FSM (RFSM).

The design layer specifies how the requirements will be implemented; the implementation layer is the FSM's code, and the test layer supports the verification development step.

Control Structure

Before focusing on the requirements FSM, it is worth discussing the control structure concept, defined by Key FSM and referenced by the requirements FSM.

An FSM model is constructed to specify some type of control function: process, behavior or dynamics of data that is sometimes called "FSM output." We

commonly see an output definition of each state virtually as a separate entity—there is nothing in the model itself that binds states' output and "keeps it all together." Key FSM adds this element—the control structure. It specifies system elements controlled or affected by the FSM, reducing error risk by requiring ALL states to define control structure values, improving the accuracy and correctness of requirements and the FSM as a whole.

Two types of control structure are defined:

• **Entities** – A list of actors and records and their properties that are controlled by the FSM. Often only one such entity is controlled by a specific FSM, but more are possible. Each of the states must explicitly define some condition over each of the controlled entities, even if it is only "no change" or "no effect" — documenting the rationale for those decisions makes them conscientious and the whole model more accurate.

• **Control variables** – a set of variables or properties (record fields) controlled by the FSM or said to be the FSM's output. Each of the FSM states must define a value for those variables, and the reasons for those values documented.

The control structure defines the scope over which the FSM has control; at the design and implementation layers, it is translated into actual objects, records and fields that are manipulated or set by the FSM.

The requirements layer

This layer contains the states' requirements specifications over the control structure and it may take one of the following three forms:

Requirement statements. Use a notation standardized for the project, product or organization to specify state requirements. These could be user stories, EARS notation, the standard "X shall do Y" or any other. These statements express system requirements for each state, but note that they should referece the FSM's control structure. If at one state we need to specify a requirement relating to entities beyond the control structure, those have to be added to the control structure, and the rest of the states have to reference it as well.

Control variable values. If the FSM's control structure comprises only simple variables, we need to specify their value at each state and nothing more. We also document the reasoning for the specified value. Examples will be: (a) session FSM —sets the session's "authenticated" variable to true or false depending if logged in or out; and set the time of entering the logged-in state (for session expiration), (b) a cruise control system's FSM, as in the example described later on, controls a few variables: boolean controlOn, real setPoint, boolean accelerating, and a few other optional boolean variables to control indications on the instruments panel.

State invariant. This form of state requirements uses a logic expression over the control structure that must evaluate "true" in the state. E.g. for a session FSM:

authenticConfidenceLevel(actualUser, user(loginName)) > C1; where the authenticConfidenceLevel and user(LoginName) functions, as well as the

threshold C1 are defined. Note that this will require performing an authentication function on the session's user—one, two or three factors depending on C1, on password complexity, etc. – that will all be part of the design, aimed at satisfying the state invariant.

Whichever requirement's notation we choose, we end up with an FSM that specifies pure requirements and their dynamics (how they change in response to events).

Requirements FSM in the Requirements Process

Figure 2 depicts the process steps involved in creating and modeling requirements with RFSM using a DFD-like diagram. Elicitation meetings (i) produce SME (subject matter expert) statements that are "raw requirements." Those are analyzed (ii) to produce draft requirements and optionally, when needed, a requirements model. An FSM is most often the choice to model dynamic behavior, process and status of application entities. And since we are dealing with requirements, RFSM can be used. (When we model requirements with an FSM and strictly limit states to represent system requirements and not design—we get RFSM de-facto.) The requirements engineer should analyze the RFSM (iii) using rules, techniques and tools designed specifically to evaluate FSM's integrity. This analysis usually produces a list of flaws; some can be resolved by correcting draft requirements and others require going back to elicitation to complete missing information or resolve conflicts. After all model analysis errors are resolved, the process moves to the final review (iv). SME's may provide additional corrections that may require going back and analyzing the model (iii) to keep its integrity. When a review passes, the requirements lose their draft status and become official. The final model (b) in its tabular form (states and state transition tables) and/or its graphical representation (state transitions diagram) is usually included in the requirements (a), specifying dynamic process or behavior that in many cases would otherwise be too complex to specify and comprehend. An additional benefit of the model is the analysis step (iii) that helps uncover hidden flaws, improving requirements quality.

FSM Analysis

An FSM model can be validated by structural and semantic rules ([1] sections IV–F and IV-G). Static and structural rules check states to be reachable, have transitions out of the state (except for terminal states), timeout, and more, and can be applied to the model directly. Semantic rules require symbolic processing, or can be tested at run-time. One rule in particular (4 - Complete transition definitions) requires a whole process to be evaluated – the event-state analysis (ESA).

Event-state analysis verifies the complete definition of the state transitions mapping function (states X events → state) that is part of the formal finite automaton and FSM definition (∂ in [3]). A concise state transitions table form—

rows representing states, columns and cells contain the next state guide the analyst in defining this function, is depicted by Table 1:

	Event 1	Event 2	…
State 1	Next state 1	Next state 3	…
State 2	Next state 2	Next state 4	…
…	…	…	…

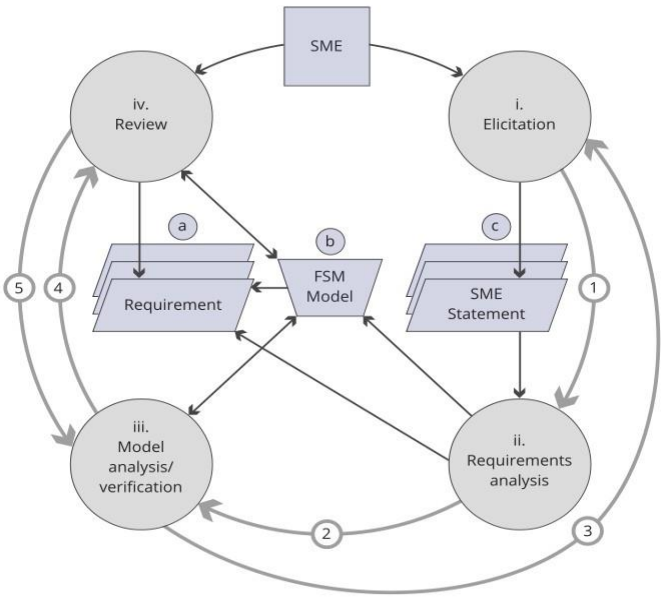


Figure 2: The FSM model in the requirements development steps

In many practical cases however, the number of states and events is too high for this form, and other forms are used (see [1]). The process checks all events against all states systematically and domain knowledge is used to add transitions and confirm where transitions are not needed, and documenting rationale for future maintenance that may require those decisions to be revisited. This simple process yields many additional transitions and sometimes new states. Combined with states' timeout, it practically guarantees that "forgotten" transitions will be added. Performed rigorously, it achieves model completeness and thus contributes to its correctness.

FSM and exceptions

Exceptions that are visible and the handling of which makes a difference to actors and stakeholders (and those constitute most of the exceptions) have to be specified at the requirements level. Most exceptions require deviation from the interaction path and can be viewed as events that trigger transitions to different states, diverting the process from the nominal path. No behavior or process model is complete, and therefore correct, without modeling such visible exceptions. Most FSM models however ignore exceptions, leaving the issue to the implementation level, where the risk of changing the design and handling them in an undesired

way is high. It is therefore required to include exceptions in the requirements model. Key FSM adds exceptions as state specific events. How those exceptions are identified is beyond the scope of this paper (for more details see [1] section IV-A-10). Identifying exceptions is part of the first step of the event-state analysis—identify all relevant events.

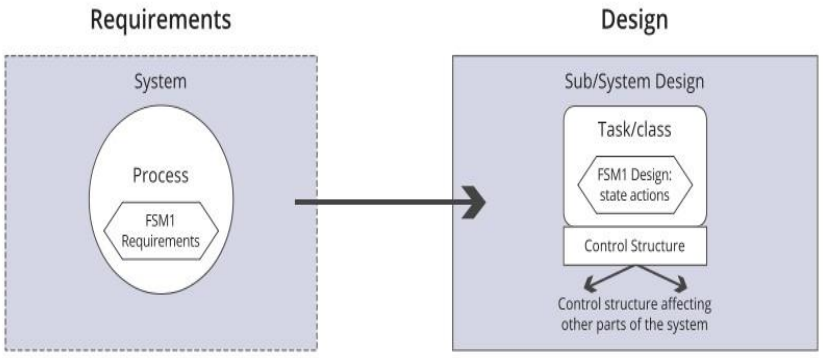
Beyond requirements

Requirements are the means for correct and successful product development, and as such they are the input for "downstream" development phases. In closing, we will explore how the requirement FSM affects the development phases that follow it.

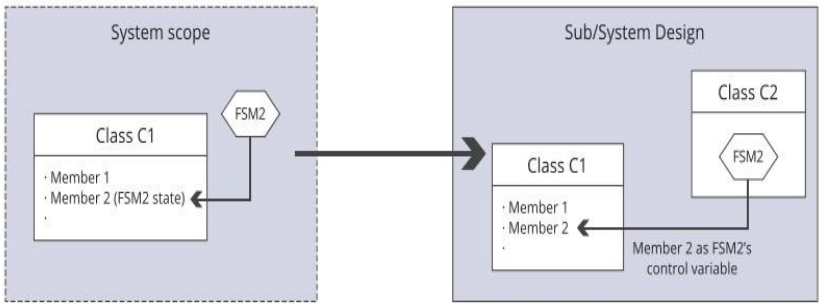
Requirements to Design

Two types of system elements are commonly modeled by an FSM: a process that switches between well-defined steps or states, and a variable that represents a state.

A common and simple design choice for a process modeled by an FSM is a dedicated module that will run an implementation of the requirements FSM—FSM1 in Figure 5. To justify this design, consider that the requirements FSM specifies a set of states and what the system is required to do at each state. Running an equivalent FSM and having its states create the conditions specified by the requirements layer is a straight-forward design that meets the requirements. This solution is the most common design choice. Figure 5-(a) illustrates this mapping of the FSM model from requirements to design.



(a) KFSM based process, requirements to design mapping



(b) A state variable class member requirements to design mapping

Figure 5: FSM based requirements transition to design

The design layer

Using the requirements layer as a reference, the design layer specifies how the requirements as specified by the requirements layer will be implemented, using a design specifications text and diagrams. A more structured way is to use the entry, do and exit actions structure, and to specify those actions for each state, so they implement the requirements layer for the state. Table 2 describes the general functionality of those actions for the various types of state requirements.

As we can see, the case of specifying states' requirements by control variable values is special—only simple entry and do actions are needed to set the required control variable values. Key FSM [1], the model that implements the layers described here, takes this case even further and its "FSM Engine" (see implementation below) identifies this case and sets the values from the FSM definition, eliminating the need for any states' action. This could actually be considered a design pattern and Figure 7 depicts such a design for our cruise control example, while Table 2 outlines how the state actions for that example change the control variables.

State requirements notation used	Entry Action	Do Action	Exit Action
Specification statements	Prepare control structure entities and variables – bring them to required initial values;	Perform any repeating action required to maintain compliance with state's requirements	Clean-up, stop any activity unique to the state
State invariant	Bring control structure to comply with state invariant	Maintain compliance with state invariant	Clean-up, stop any activity unique to the state
Control variable values	Set values	Changing values → Update	_

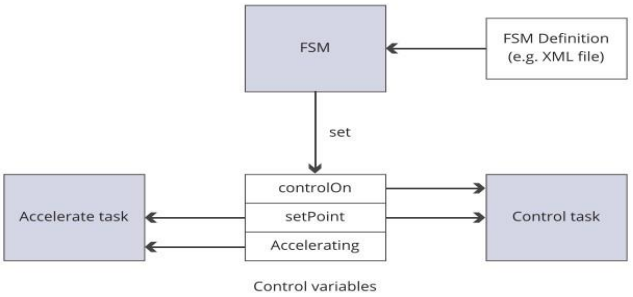


Figure 7: An FSM based design of the cruise control system

Cruise Control: control variable values set by state actions			
State	Entry Action	Do Action	Exit Action

No control	controlOn = false;
	setPoint – no change
	accelerating = false;		

Constant speed	controlOn = true;
	setPoint – no change-
	accelerating = false;
			-

Decelerating	controlOn = false;
	setPoint = current speed
	accelerating = false;
		setPoint = current speed	

Accelerating	controlOn = true; accelerating = true;	setPoint = setPoint + (t-t0)*DV/Dt	

MITRC

LECTURE NO:-7

Data Flow Diagram

Software analysis and design includes all activities, which help the transformation of requirement specification into implementation. Requirement specifications specify all functional and non-functional expectations from the software. These requirement specifications come in the shape of human readable and understandable documents, to which a computer has nothing to do.

Software analysis and design is the intermediate stage, which helps human-readable requirements to be transformed into actual code.

Let us see few analysis and design tools used by software designers:

Data Flow Diagram

Data flow diagram is graphical representation of flow of data in an information system. It is capable of depicting incoming data flow, outgoing data flow and stored data. The DFD does not mention anything about how data flows through the system.

There is a prominent difference between DFD and Flowchart. The flowchart depicts flow of control in program modules. DFDs depict flow of data in the system at various levels. DFD does not contain any control or branch elements.

Types of DFD

Data Flow Diagrams are either Logical or Physical.

- Logical DFD - This type of DFD concentrates on the system process, and flow of data in the system.For example in a Banking software system, how data is moved between different entities.

- Physical DFD - This type of DFD shows how the data flow is actually implemented in the system. It is more specific and close to the implementation.

DFD Components

DFD can represent Source, destination, storage and flow of data using the following set of components -



• Entities - Entities are source and destination of information data. Entities are represented by a rectangles with their respective names.

• Process - Activities and action taken on the data are represented by Circle or Round-edged rectangles.

• Data Storage - There are two variants of data storage - it can either be represented as a rectangle with absence of both smaller sides or as an open-sided rectangle with only one side missing.

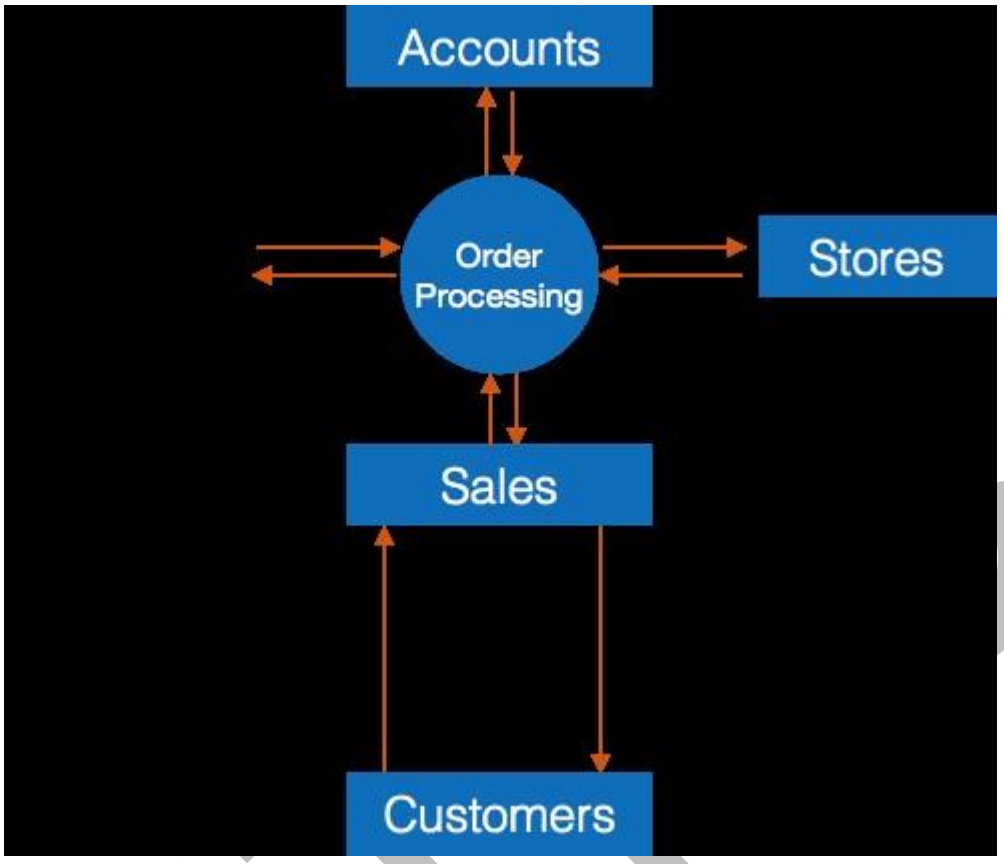
• Data Flow - Movement of data is shown by pointed arrows. Data movement is shown from the base of arrow as its source towards head of the arrow as destination.

Levels of DFD

• Level 0 - Highest abstraction level DFD is known as Level 0 DFD, which depicts the entire information system as one diagram concealing all the underlying details. Level 0 DFDs are also known as context level DFDs.



• Level 1 - The Level 0 DFD is broken down into more specific, Level 1 DFD. Level 1 DFD depicts basic modules in the system and flow of data among various modules. Level 1 DFD also mentions basic processes and sources of information.



• Level 2 - At this level, DFD shows how data flows inside the modules mentioned in Level 1.

Higher level DFDs can be transformed into more specific lower level DFDs with deeper level of understanding unless the desired level of specification is achieved.

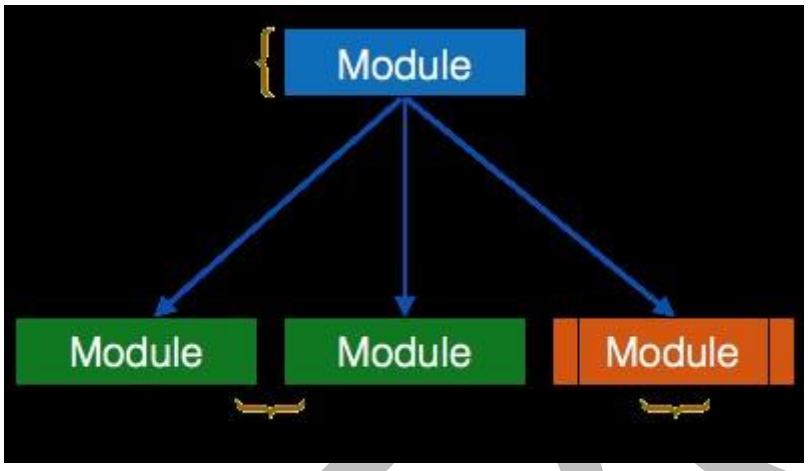
Structure Charts

Structure chart is a chart derived from Data Flow Diagram. It represents the system in more detail than DFD. It breaks down the entire system into lowest functional modules, describes functions and sub-functions of each module of the system to a greater detail than DFD.

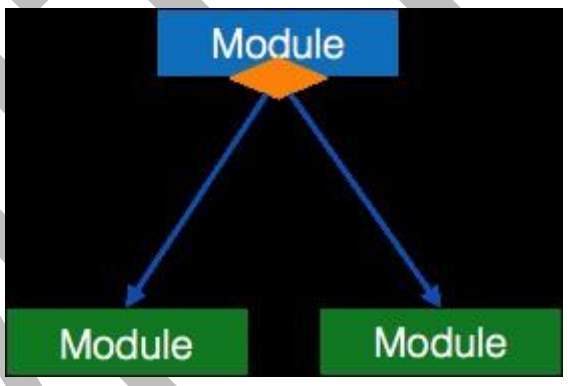
Structure chart represents hierarchical structure of modules. At each layer a specific task is performed.

Here are the symbols used in construction of structure charts -

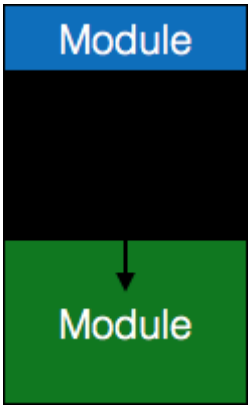
• Module - It represents process or subroutine or task. A control module branches to more than one sub-module. Library Modules are re-usable and invokable from any module.



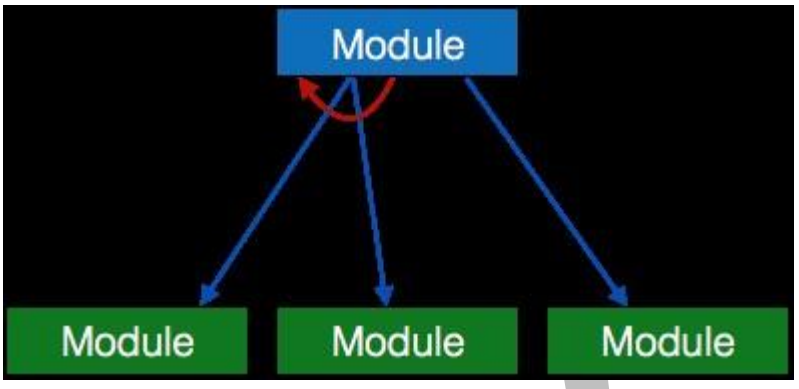
• Condition - It is represented by small diamond at the base of module. It depicts that control module can select any of sub-routine based on some condition.



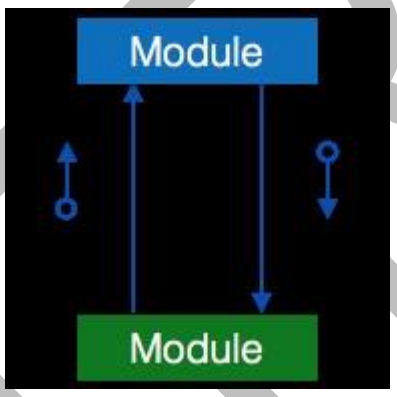
• Jump - An arrow is shown pointing inside the module to depict that the control will jump in the middle of the sub-module.



• Loop - A curved arrow represents loop in the module. All sub-modules covered by loop repeat execution of module.



• Data flow - A directed arrow with empty circle at the end represents data flow.



• Control flow - A directed arrow with filled circle at the end represents control flow.

MITRC

LECTURE NO:-8

Control Specification Behavioral Modeling

Control Specification Behavioral Modeling in software design is the technical kernel of the software engineering process.

Software design is a process through which the requirements are translated into a representation of software. Initially the representation depicts a holistic view of software. Subsequent refinement leads to a representation that is close to source code.

The software design process has technical and management aspects:

a) from a project management point of view, software design is conducted in two steps:

- preliminary design: concerns the transformation of requirements into data and software architecture;

- detailed design: focuses on refinements to the architectural representation that lead to detailed data structure and algorithmic representations of software;

b) from a technical point of view, software design involves the following steps:

- data design: transforms the information domain model created during the analysis into interaction.

data structures that will be required to implement the software;

- architectural design: defines the relationship among major structural components of the program;

- procedural design: transforms the structural components into a procedural description of the software;

- interface design: establishes the layout and interaction mechanisms for human-machine

1.2 Fundamental Concepts

The software designer raises the following questions:

- what criteria can be used to partition the software into individual components?

- how is function or data structure detail separated from a conceptual representation of the software?

- are there uniform criteria that define technical quality of software design?

The basic concepts of the software design process are:

- abstraction

- refinements

- modularity

- architecture

- hierarchy

- data structure

- software procedures

- information hiding

2. Object-Oriented vs. Procedural Design

2.1. Procedural Design

The procedural design is often understood as a software design process that uses mainly control commands such as: sequence, condition, repetition, which are applied to the predefineddata.

Sequences serve to achieve the processing steps in order that is essential in the specification of any algorithm.

Conditions provide facilities for achieving selected processing according to some logical statement.

Repetitions serve to achieve loopings during the computation process.

These three commands are implemented as ready programming language constructs.

The programming languages that provide such command constructs are called imperative programming languages.

The software design technique that relies on these constructs is called procedural design, or also structured design.

MITRC

2.2. Object-Oriented Design

2.2.1. The Macro Design Process

The macro design process is closely related to the **traditional waterfall life cycle**, and serves as controlling framework for the micro process.

The basic philosophy of the macro process is that of incremental development.

The macro design process tends to track the following activities:

Conceptualization: includes establishing the core requirements to the software.

The conceptualization may consist of the following activities:

- assemble an appropriate team to develop a prototype;

- develop a prototype;

- evaluate the prototype.

The prototypes constructed at this stage are meant to be thrown away. Such prototypes are not to be allowed to evolve directly into a system. At this stage any programming paradigm may be used.

Analysis: the purpose of the analysis is to provide a description of a problem.

The description must be complete, consistent, readable, and reviewable. The primary activities during the analysis are:

- identify all primary function points of the system, if possible group them

into clusters of functionally related behaviours; Function points are the outwardly observable and testable behaviours of the system. From the perspective of the user, a function point represents some primary activity of the system in response to some event.

- for each set of function points storyboard a scenario using use-cases;

- develop the use-case view of the system.

During the analysis we seek to model the world by classes from the vocabulary of the problem domain.

MITRC

Design: the purpose of the design is to create the architecture of the system The primary activities during the design are:

- allocate the clusters of function points to layers and partitions of the architecture;

- perform one spin of corresponding micro design subprocesses !

- validate the architecture by creating an executable release;

- asses the risk of each architectural component.

After the global system construction, release planning sets. A typical order of events for this activity is as follows:

- given the architectural components organize them in order of behaviors;

- allocate the relevant function points to series of architectural releases; Elaborate the Component Diagram and develop the Component view of the system;

- develop the Deployment Diagram and the Deployment view of the system.

- perform task planning, when the work breakdown structure is ready. A product of release planning is a formal development plan, which identifies the stream of architectural releases, team tasks, and risk assessments.

Evolution: this is essentially the process of implementing the system

The primary activities during the evolution process are:

- assign tasks to the team to carry out this release;

- initiate one spin of corresponding micro design subprocesses !

- supervise the micro process by establishing appropriate reviews;

- force closure of the micro process by making an executable release.

In practice the following kinds of changes are to be expected during the system evolution:

- adding new classes to the system;

- changing the implementation of a class;

MITRC

- changing the representation of a class;

- changing the interface of a class.

Maintenance: this is the activity of managing postdelivery evolution of the system.

The primary activities during the maintenance are:

- prioritize the requests for major enhancement or bug reports that denote systematic problems, and asses the cost of development;

- establish a meaningful collection of these changes and treat them as function points for next elaboration;

- if resources allow it, add more localized enhancements to the next release;

- manage the next evolutionary release.

2.2.2 The Micro Design Process

The micro design process is closely related to the **spiral model** of software development, and serves as framework for iterative and incremental approach to development.

Identifying Classes and Objects: the purpose is to establish the boundaries of the problem.

The activities at this process are:

- apply the classical approach to object-oriented development to generate a set of candidate classes and objects. The abstractions should be directly related to the function points.

- develop Class and Object Diagrams ! For each abstraction do the following:

- enumerate its roles and responsibilities;

- devise a sufficient set of operations that satisfy these responsibilities;

- consider each operation in tern and ensure that it is primitive. If not, isolate and expose its more primitive operations.

MITRC

- consider the needs for construction, destruction and copying;

- consider the need for completeness: add other primitive operations that are not necessarily required by the immediate clients, and would probably be used by future clients.

MITRC

LECTURE NO:-9

Process Specification Behavioral Modeling

Software design is the technical kernel of the software engineering process.

Software design is a process through which the requirements are translated into a representation of software. Initially the representation depicts a holistic view of software. Subsequent refinement leads to a representation that is close to source code.

The software design process has technical and management aspects:

a) from a project management point of view, software design is conducted in two steps:

- preliminary design: concerns the transformation of requirements into data and software architecture;

- detailed design: focuses on refinements to the architectural representation that lead to detailed data structure and algorithmic representations of software;

b) from a technical point of view, software design involves the following steps:

- data design: transforms the information domain model created during the analysis into interaction.

data structures that will be required to implement the software;

- architectural design: defines the relationship among major structural components of the program;

- procedural design: transforms the structural components into a procedural description of the software;

- interface design: establishes the layout and interaction mechanisms for human-machine

1.2 Fundamental Concepts

The software designer raises the following questions:

- what criteria can be used to partition the software into individual components?

- how is function or data structure detail separated from a conceptual representation of the software?

- are there uniform criteria that define technical quality of software design?

The basic concepts of the software design process are:

- abstraction

- refinements

- modularity

- architecture

- hierarchy

- data structure

- software procedures

- information hiding

2. Object-Oriented vs. Procedural Design

2.1. Procedural Design

The procedural design is often understood as a software design process that uses mainly control commands such as: sequence, condition, repetition, which are applied to the predefineddata.

Sequences serve to achieve the processing steps in order that is essential in the specification of any algorithm.

Conditions provide facilities for achieving selected processing according to some logical statement.

Repetitions serve to achieve loopings during the computation process.

These three commands are implemented as ready programming language constructs.

The programming languages that provide such command constructs are called imperative programming languages.

The software design technique that relies on these constructs is called procedural design, or also structured design.

MITRC

2.2. Object-Oriented Design

2.2.1. The Macro Design Process

The macro design process is closely related to the **traditional waterfall life cycle**, and serves as controlling framework for the micro process.

The basic philosophy of the macro process is that of incremental development.

The macro design process tends to track the following activities:

Conceptualization: includes establishing the core requirements to the software.

The conceptualization may consist of the following activities:

- assemble an appropriate team to develop a prototype;

- develop a prototype;

- evaluate the prototype.

The prototypes constructed at this stage are meant to be thrown away. Such prototypes are not to be allowed to evolve directly into a system. At this stage any programming paradigm may be used.

Analysis: the purpose of the analysis is to provide a description of a problem.

The description must be complete, consistent, readable, and reviewable. The primary activities during the analysis are:

- identify all primary function points of the system, if possible group them

into clusters of functionally related behaviours; Function points are the outwardly observable and testable behaviours of the system. From the perspective of the user, a function point represents some primary activity of the system in response to some event.

- for each set of function points storyboard a scenario using use-cases;

- develop the use-case view of the system.

During the analysis we seek to model the world by classes from the vocabulary of the problem domain.

MITRC

Design: the purpose of the design is to create the architecture of the system The primary activities during the design are:

- allocate the clusters of function points to layers and partitions of the architecture;

- perform one spin of corresponding micro design subprocesses !

- validate the architecture by creating an executable release;

- asses the risk of each architectural component.

After the global system construction, release planning sets. A typical order of events for this activity is as follows:

- given the architectural components organize them in order of behaviors;

- allocate the relevant function points to series of architectural releases; Elaborate the Component Diagram and develop the Component view of the system;

- develop the Deployment Diagram and the Deployment view of the system.

- perform task planning, when the work breakdown structure is ready. A product of release planning is a formal development plan, which identifies the stream of architectural releases, team tasks, and risk assessments.

Evolution: this is essentially the process of implementing the system

The primary activities during the evolution process are:

- assign tasks to the team to carry out this release;

- initiate one spin of corresponding micro design subprocesses !

- supervise the micro process by establishing appropriate reviews;

- force closure of the micro process by making an executable release.

In practice the following kinds of changes are to be expected during the system evolution:

- adding new classes to the system;

- changing the implementation of a class;

MITRC

- changing the representation of a class;

- changing the interface of a class.

Maintenance: this is the activity of managing postdelivery evolution of the system.

The primary activities during the maintenance are:

- prioritize the requests for major enhancement or bug reports that denote systematic problems, and asses the cost of development;

- establish a meaningful collection of these changes and treat them as function points for next elaboration;

- if resources allow it, add more localized enhancements to the next release;

- manage the next evolutionary release.

2.2.2 The Micro Design Process

The micro design process is closely related to the **spiral model** of software development, and serves as framework for iterative and incremental approach to development.

MITRC

LECTURE NO:-10

Extension for data intensive applications

EXTENSIONS FOR DATA—INTENSIVE APPLICATIONS

The basic notation for structured analysis works well when relatively simple information flows through a series of processes. As such in many information systems applications, there is a need to represent the relationship between complex collections of data. To accomplish this, structured analysis notation has been extended to encompass a data modeling component.

Data modeling answers a set of specific questions that are relevant to any data processing application.

- What are the primary data objects to be processed by the system ?
- What is the composition of each data object and what attributes describe the object?
- Where do the objects currently reside '?
- What are the relationships between each object and other objects ?
- What is the relationship between the objects and the processes that transform them ?

To answer these questions, data modeling methods make use of the entity—relationship (E—R) diagram. In the context of structured analysis, the E—R diagram provides additional insight into the detail of data stores and their relationship to processes within the flow model. In addition, the E—R diagram complements the representation of data content contained in the requirements dictionary.

MITRC